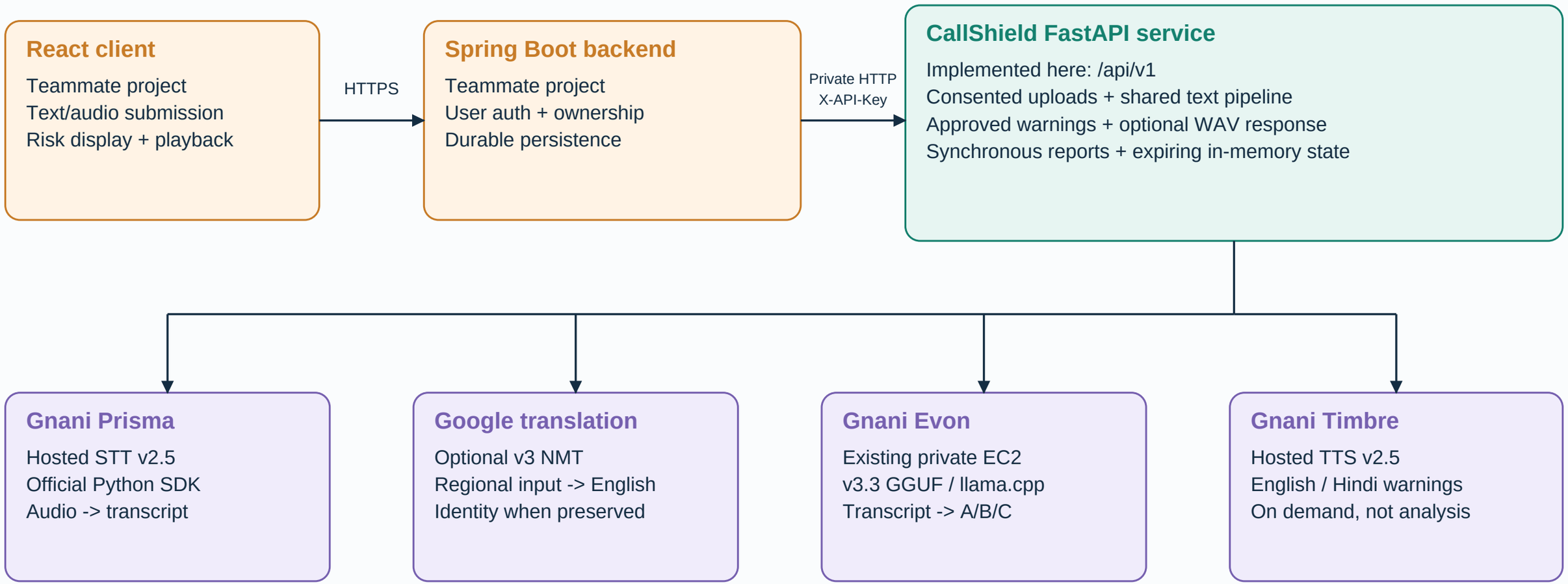
System context & ownership

The Python service is the implemented product in this repository. Spring owns the end-user boundary.



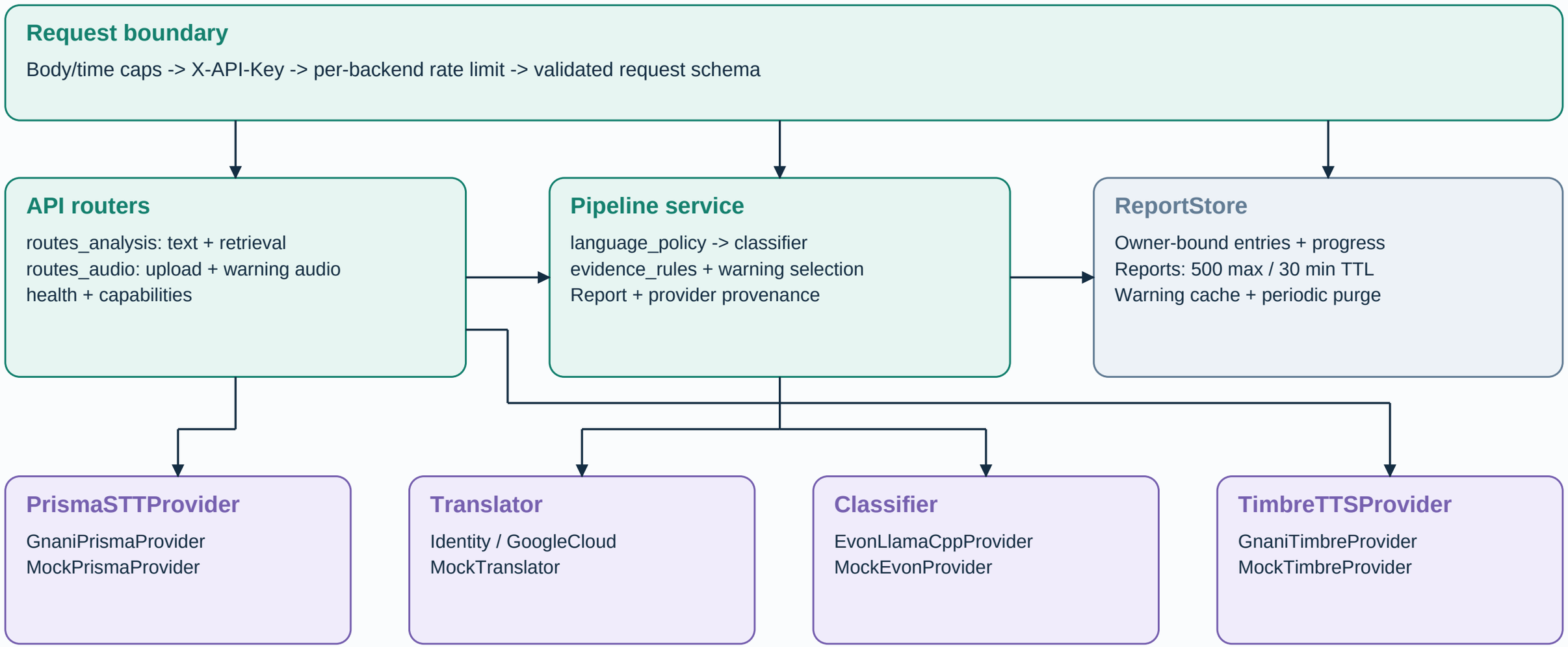
Legend Implemented Python | Real provider boundary | Separate teammate project

Sources: app/main.py | app/core/auth.py | app/services/pipeline.py | SPRING_BOOT_HANDOFF.md

Source snapshot: 20 October 2026 | M0-M5 implemented | External clients are separate projects

Python components & adapter seams

Routes orchestrate services; provider details remain behind small, replaceable contracts.



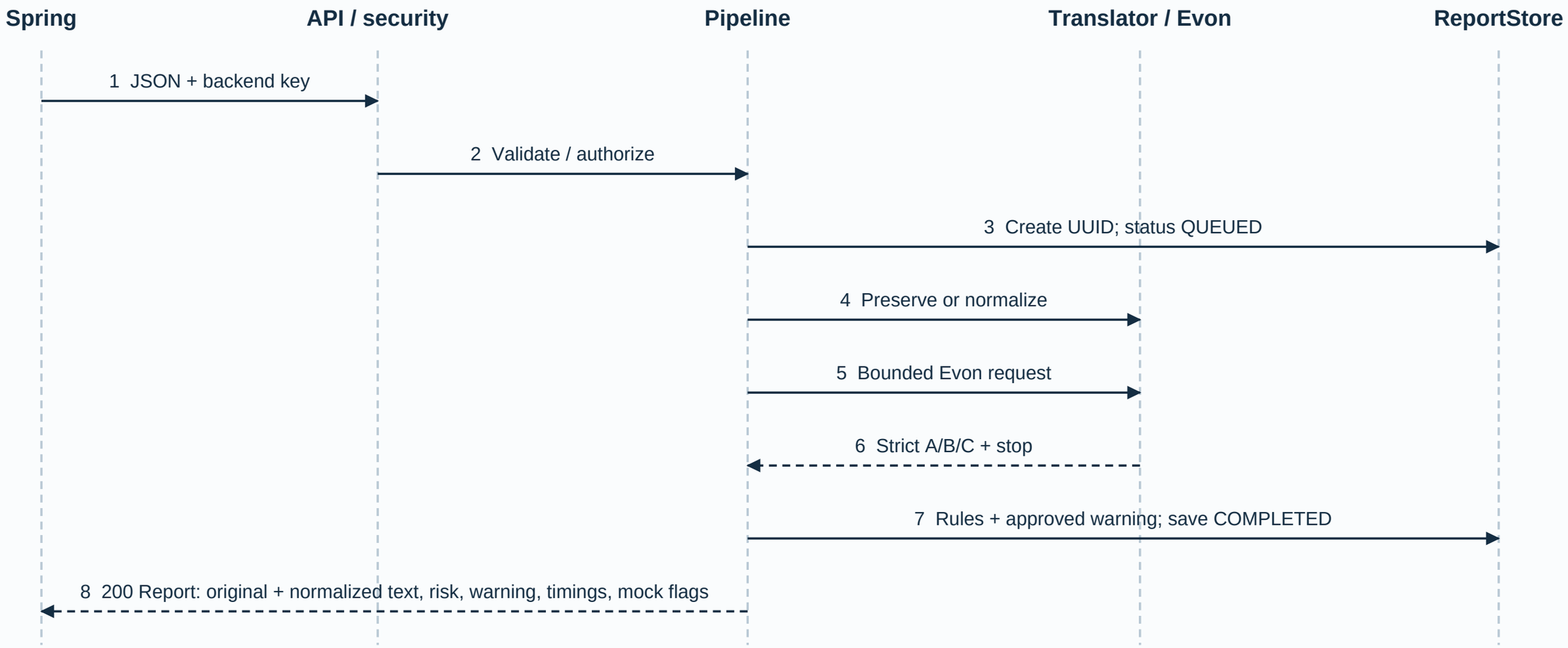
Composition: create_app() selects mock or real providers at startup; shared contracts keep the REST API stable.

Sources: app/api/* | app/services/* | app/providers/interfaces.py | app/main.py

Source snapshot: 20 October 2026 | M0-M5 implemented | External clients are separate projects

Text analysis: synchronous sequence

POST /api/v1/analyze/text returns a completed report or an honest operational failure.



Failure contract

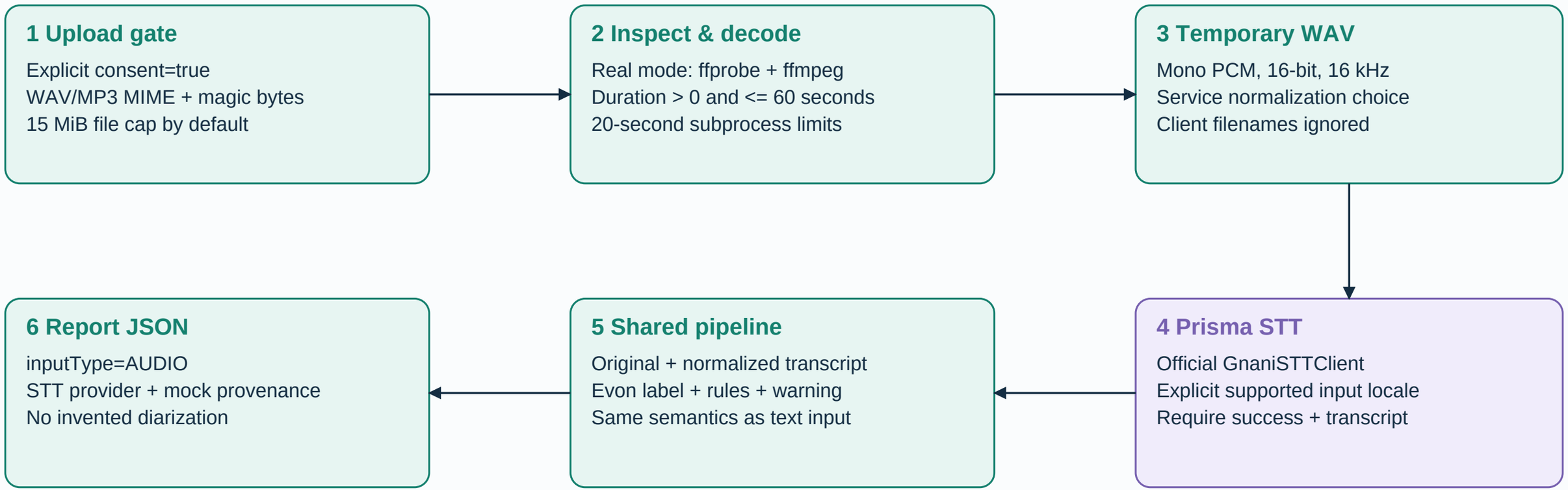
Invalid/empty model output -> 502; unavailable Evon -> 503; timeout -> 504; queue pressure -> 429. No false LOW_RISK result.

Sources: app/api/routes_analysis.py | app/services/pipeline.py | app/providers/evon_llamacpp.py

Source snapshot: 20 October 2026 | M0-M5 implemented | External clients are separate projects

Consented audio -> shared analysis

POST /api/v1/analyze/audio adds safe media handling before the same text pipeline.



Temporary inputs are deleted on success and failure

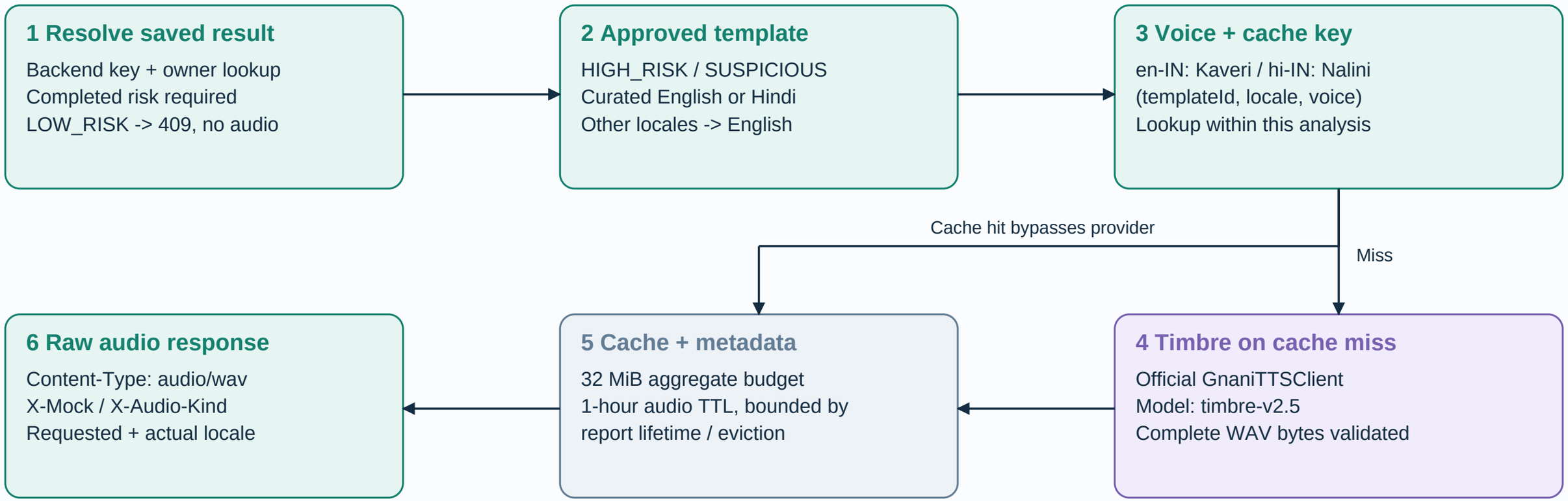
Mock WAV validation uses the standard library; mock STT returns a labeled fixed fixture. Real input always uses the media tools. MP3 needs FFmpeg in either mode.

Sources: app/api/routes_audio.py | app/services/audio_utils.py | app/providers/gnani_prisma.py

Source snapshot: 20 October 2026 | M0-M5 implemented | External clients are separate projects

Warning audio is a separate operation

POST /api/v1/analyses/{id}/warning-audio accepts language/voice, never arbitrary speech text.



TTS failure never discards classification

The report remains COMPLETED with its existing risk; warning.audioStatus becomes FAILED. Mock output is a labeled 440 Hz tone, not synthesized speech.

Sources: app/api/routes_audio.py | app/services/warnings.py | app/providers/gnani_timbre.py | report_store.py

Source snapshot: 20 October 2026 | M0-M5 implemented | External clients are separate projects

Mock mode & real integration contracts

The same REST contract is served by different startup-selected adapters. No silent fallback exists.

Startup configuration

MOCK_MODE=true by default | Real/production mode requires a backend API key with at least 24 characters

MOCK_MODE=true

Evon: deterministic demonstration rules
Prisma: fixed English scam transcript
Timbre: playable 0.5-second test tone
Translation: identity, or fixed mock fixture
Every mocked stage has mock=true
No provider network calls or API credentials

MOCK_MODE=false

Evon: private /v1/chat/completions HTTP
Prisma/Timbre: gnani-vachana 0.8.1
Speech auth: X-API-Key-ID via official SDK
Translation: identity or Google v3 NMT
Speech SDK calls: bounded worker threads
Errors stay explicit; predictions are not fabricated

Evon request invariants

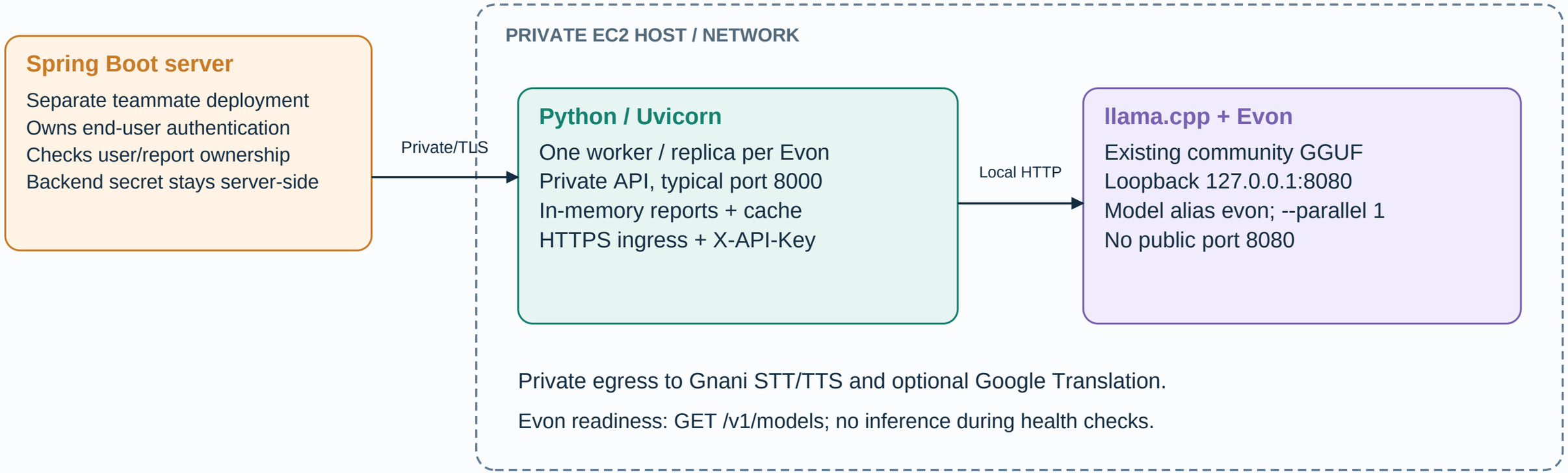
enable_thinking=false; temperature=0; max_tokens=8; stream=false. Accept only A/B/C after whitespace with finish_reason=stop. Ignore reasoning_content; confidence=null.

Sources: app/main.py | app/providers/mock_providers.py | gnani_prisma.py | gnani_timbre.py | google_translation.py

Source snapshot: 20 October 2026 | M0-M5 implemented | External clients are separate projects

Deployment & trust boundaries

Recommended topology: Python and llama.cpp on the same EC2 host; expose only the Python private ingress.



Local Docker option

Compose defaults to mock mode; host 127.0.0.1:8000.
Non-root image, read-only compose root + tmpfs.
Container loopback is not the EC2 host loopback.

Operational constraints

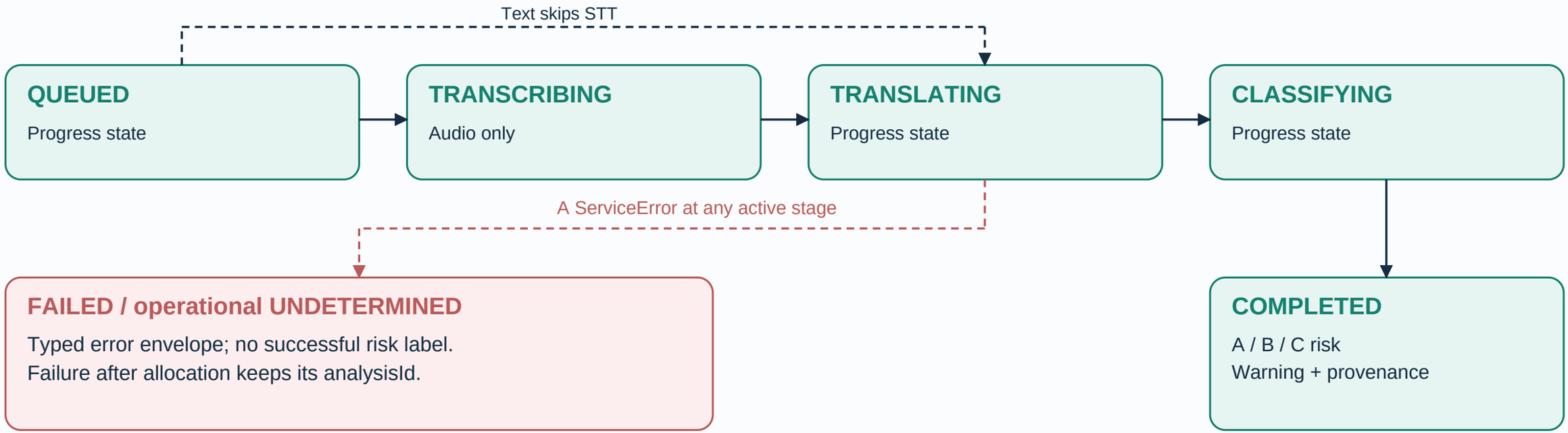
Evon stopped -> readiness/classification 503.
Configured speech readiness is not a quota/auth probe.
No database or distributed queue is implemented.

Sources: Dockerfile | docker-compose.yml | app/config.py | app/core/auth.py | app/api/routes_health.py

Source snapshot: 20 October 2026 | M0-M5 implemented | External clients are separate projects

Result lifecycle & teammate contract

IDs refer to real in-memory entries; Spring should retain successful reports durably.



Stable report payload

analysisId + status + inputType
Original and normalized text
Risk: code, level, null confidence
Rules + warning + provenance
Timing fields + mock flags

GET /analyses/{id}

200: report, progress or saved error
404: missing / expired entry
Check body status even on HTTP 200
500 entries / 30-minute report TTL
Process restart loses all entries

Spring integration rules

Never translate failures into LOW_RISK
Keep backend key away from React
Authorize ownership before proxying
No automatic paid-request retries
Fetch capabilities before UI setup

Sources: app/services/report_store.py | app/services/pipeline.py | app/api/routes_analysis.py | schemas/responses.py

Source snapshot: 20 October 2026 | M0-M5 implemented | External clients are separate projects